

École Supérieure en Informatique de Sidi Bel Abbès (ESI-SBA) | Spécialité
ISI — 2^{ème} année Cycle Supérieur

Module: Smart Home & Assistive Robotics

Autonomous Object Retrieval Pipeline

Navigation · Alignment · Grasping



Supervised by **Pr. Rahmoune** | Academic Year 2024–2025



Contents

Contents	1
1 System Architecture Overview	3
1.1 Introduction and Motivation	3
1.2 Distributed Node Architecture	3
1.2.1 Communication Protocols	4
2 Computer Vision and Object Detection	5
2.1 Design Philosophy: Classical CV over Deep Learning	5
2.2 The Full Image Processing Pipeline	5
2.2.1 Stage 1 — Gaussian Blurring	5
2.2.2 Stage 2 — HSV Colour Thresholding	6
2.2.3 Stage 3 — Morphological Operations	6
2.2.4 Stage 4 — Contour Extraction and Spatial Moments	6
3 Automated Navigation and Alignment	8
3.1 Overview	8
3.2 Error Calculation	8
3.3 Dead Zone and Hunting Prevention	8
3.4 Proportional Pulse Control	9
4 Distance Estimation and Approach Phase	10
4.1 Monocular Depth via Pixel Area	10
4.2 ZUPT-Inspired Stall Detection and Recovery	10
4.2.1 The Stall Problem	10
4.2.2 Rolling-Window Detection	10
4.2.3 Escalating Recovery Protocol	11
5 Robotic Arm Kinematics and Manipulation	12
5.1 Architecture: Pre-Calibrated Poses vs. Dynamic IK	12
5.2 Joint Convention	12
5.3 The Manipulation Sequence	13
5.3.1 State 1 — Stow (Navigation Mode)	13
5.3.2 State 2 — Extend (Elevated Approach)	13
5.3.3 State 3 — Open and Grab	13
5.3.4 State 4 — Lift and Secure	13
6 Hardware Abstraction Layer	15

6.1	The Smooth Motion Problem	15
6.2	arm_servo_listener.py: The pigpio HAL	15
6.2.1	smooth_move() — Linear Interpolation Profile	15
6.3	PWM Signal Specification	16
7	Full System Integration and State Machine	17
7.1	Master State Machine	17
7.2	Key Design Decisions Summary	18
8	Hardware Specifications	19
9	Conclusion and Perspectives	20
9.1	Summary	20
9.2	Perspectives and Future Work	20

System Architecture Overview

§1.1 Introduction and Motivation

Modern assistive robotics requires the seamless integration of computer vision, control theory, and precise mechanical actuation. The **Autonomous Object Retrieval Pipeline** is the core intelligence layer of the *Smart Home & Assistive Robot System*. Its primary mission is to locate a target object anywhere in the robot's field of view, autonomously navigate to it, and physically retrieve it without human intervention.

This pipeline is designed around a strict constraint: **computational frugality**. Rather than relying on GPU-accelerated deep learning or expensive depth sensors, every stage of the pipeline is engineered to run reliably on edge hardware — a Raspberry Pi 3 — using classical computer vision and deterministic control strategies.

§1.2 Distributed Node Architecture

The full system is distributed across two computational nodes, each responsible for a distinct class of tasks:

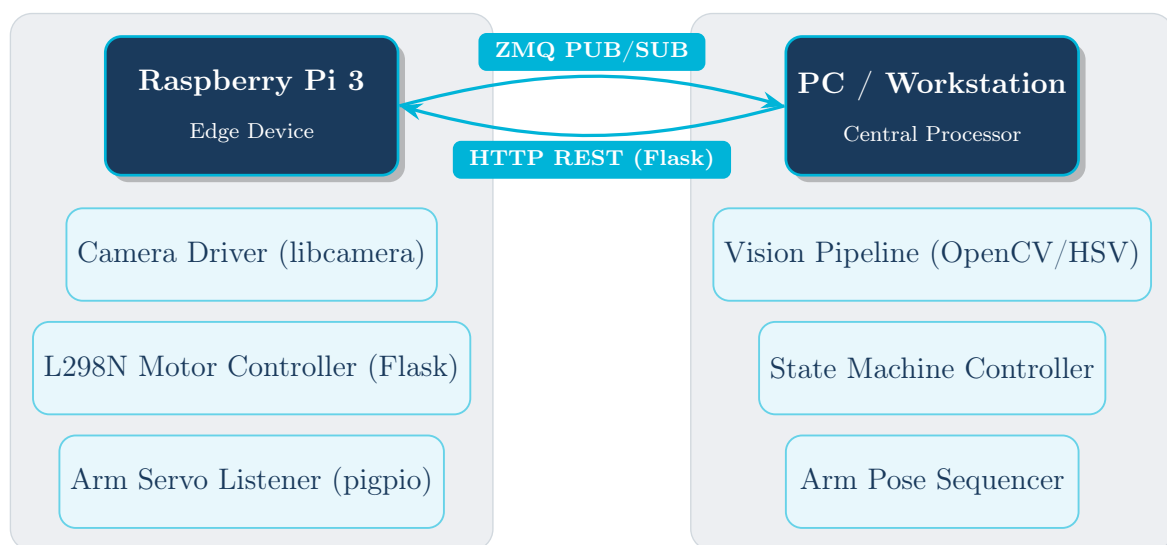


Figure 1.1: Distributed two-node architecture of the retrieval pipeline.

1.2.1 Communication Protocols

Two distinct protocols are used, each selected for a specific reason:

Protocol	Direction	Rationale
ZMQ PUB/SUB	Pi → PC	High-throughput, low-latency video streaming. The CONFLATE socket option ensures the PC always processes the <i>latest</i> frame, dropping stale ones to avoid pipeline lag.
ZMQ PUB/SUB	PC → Pi	Arm pose commands sent to the <code>arm_servo_listener</code> running on the Pi.
HTTP REST (Flask)	PC → Pi	Motor control commands (<code>/forward</code> , <code>/left</code> , <code>/stop</code> , etc.). Stateless HTTP provides built-in failure isolation — a missed command does not corrupt subsequent state.

Table 1.1: Communication protocols and their design rationale.

Why CONFLATE on the Video Socket?

Without **CONFLATE**, ZMQ queues every published frame. If the vision pipeline is briefly slower than the camera (e.g., during a complex morphological pass), a backlog of old frames builds up. **CONFLATE** replaces the queued message with the newest one, guaranteeing the controller always reacts to *current* reality, not 2-second-old frames.

Computer Vision and Object Detection

§2.1 Design Philosophy: Classical CV over Deep Learning

The vision pipeline deliberately avoids neural network object detectors (YOLO, SSD, etc.). On a Raspberry Pi 3, inference for such models takes 300–800 ms per frame, yielding an effective control loop of barely 1–2 Hz. Classical HSV thresholding runs in under 5 ms on the same hardware — a 100× speed advantage that directly translates to more responsive navigation.

The trade-off: HSV detection is target-specific (calibrated per object color) and sensitive to illumination changes. Both are addressed explicitly in the pipeline design.

§2.2 The Full Image Processing Pipeline

Every incoming frame passes through a deterministic five-stage pipeline before a usable centroid is produced:

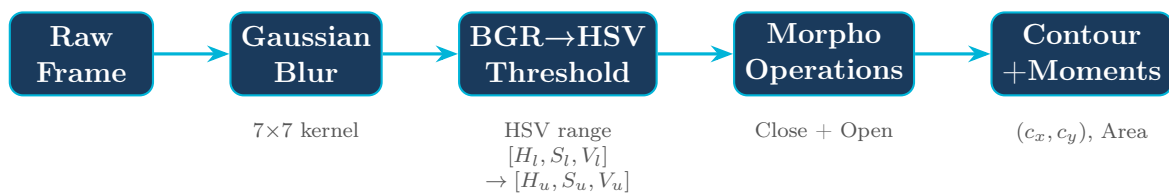


Figure 2.1: Five-stage image processing pipeline.

2.2.1 Stage 1 — Gaussian Blurring

A 7×7 Gaussian kernel is convolved with the raw frame to attenuate high-frequency noise (sensor grain, reflections, fine textures). The kernel size was chosen empirically: a 3×3 kernel is too weak to suppress specular highlights on shiny objects; a 9×9 kernel blurs colour boundaries enough to cause centroid drift in small objects.

2.2.2 Stage 2 — HSV Colour Thresholding

The blurred frame is converted from BGR (OpenCV’s native channel order) to the HSV (Hue-Saturation-Value) colour space using `cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)`.

Why HSV rather than BGR/RGB?

In BGR/RGB space, a *colour* changes all three channel values simultaneously as lighting changes. In HSV, hue (the “colour identity”) is separated from brightness. A white light source makes an object brighter, but its **H** channel stays roughly constant. This makes HSV thresholds far more robust across different lighting conditions.

A lower bound $[H_l, S_l, V_l]$ and upper bound $[H_u, S_u, V_u]$ define the target colour range. For **black** objects, hue is irrelevant: the mask simply thresholds the Value channel below a brightness ceiling ($V < V_{threshold}$), making detection illumination-independent.

The binary mask M produced is:

$$M(x, y) = \begin{cases} 255 & \text{if } H_l \leq H(x, y) \leq H_u \text{ and } S_l \leq S \leq S_u \text{ and } V_l \leq V \leq V_u \\ 0 & \text{otherwise} \end{cases}$$

2.2.3 Stage 3 — Morphological Operations

Raw binary masks always contain noise: small isolated *foreground blobs* from background reflections, and small *holes* inside the object from specular highlights. Two sequential operations clean the mask:

1. **Morphological Closing** (dilation $\oplus$ then erosion $\ominus$): fills small holes and smooth contour gaps *within* the object.
2. **Morphological Opening** (erosion $\ominus$ then dilation $\oplus$): removes small isolated noise blobs from the background.

Both use a rectangular structuring element of size 5×5 pixels. The ordering (close first, open second) is critical: reversing it would first remove corner details of the target, then attempt to fill the gaps it just created.

2.2.4 Stage 4 — Contour Extraction and Spatial Moments

`cv2.findContours()` extracts the outer boundary polygons from the cleaned mask. The pipeline selects the **largest contour by area** (using `cv2.contourArea()`) — the assumption being that the calibrated target will always be the dominant coloured object in the scene.

From this largest contour, **image moments** are computed:

$$m_{pq} = \sum_x \sum_y x^p y^q M(x, y)$$

The geometric centroid — the visual “centre of mass” of the object projection — is:

$$c_x = \frac{m_{10}}{m_{00}}, \quad c_y = \frac{m_{01}}{m_{00}}$$

where m_{00} is the zero-order moment (total foreground pixel count), directly used as the **apparent pixel area** for distance estimation.

Automated Navigation and Alignment

§3.1 Overview

Before any grasping attempt, the robot must face the target object precisely head-on. This is essential for the arm’s fixed pre-calibrated reach to intersect the object reliably. The alignment system is a **closed-loop proportional controller** operating entirely in image space — no IMU, no odometry.

§3.2 Error Calculation

Given the detected centroid c_x and the frame half-width $W/2$ (the camera’s optical axis projected onto the image), the lateral alignment error is:

$$E_x = c_x - \frac{W}{2}$$

- $E_x > 0$: object is to the right of centre $\Rightarrow$ rotate clockwise.
- $E_x < 0$: object is to the left of centre $\Rightarrow$ rotate counter-clockwise.
- $E_x \approx 0$: object is centred $\Rightarrow$ stop rotating.

§3.3 Dead Zone and Hunting Prevention

A naive zero-error target causes the robot to oscillate indefinitely around the centre (*hunting*). A **dead zone** of ± 40 pixels eliminates this:

$$\text{Aligned} \iff |E_x| < 40 \text{ px}$$

When $|E_x| < 40$, no rotation command is issued. To filter transient false alignments (vibrations, jitter), the system requires the object to remain within the dead zone for **three consecutive frames** before the “aligned” state is confirmed and the approach phase begins.

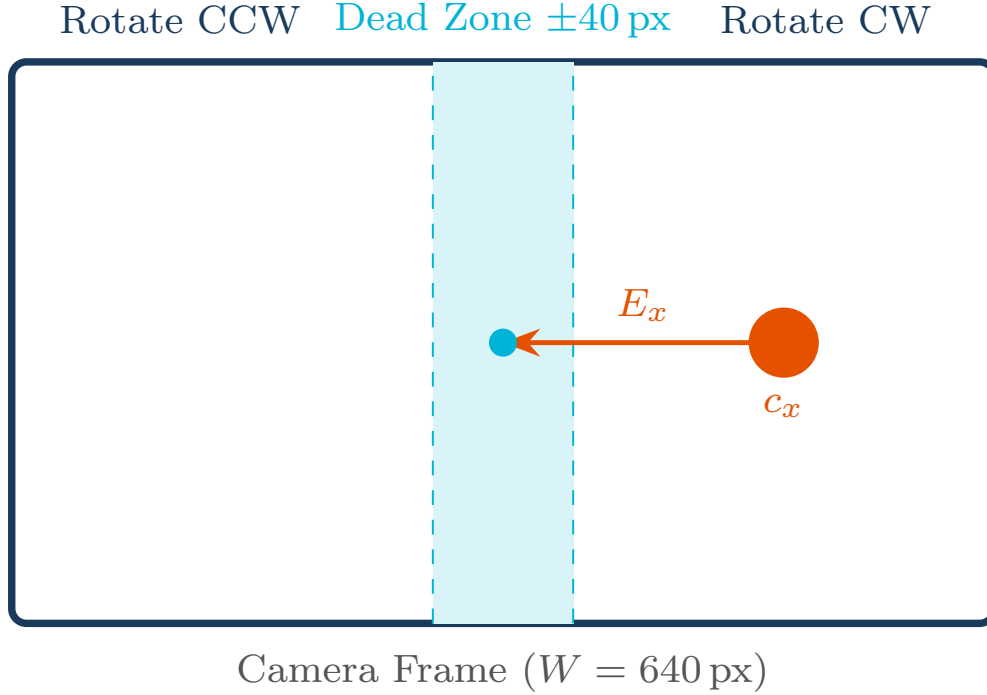


Figure 3.1: Lateral alignment error E_x and the ± 40 px dead zone.

§3.4 Proportional Pulse Control

When $|E_x| \geq 40$, the motor controller issues a rotation pulse of duration T_{pulse} (in milliseconds):

$$T_{\text{pulse}} = k_p \times |E_x|$$

where k_p is a tuning gain. This is a **proportional controller in time**: large errors produce long pulses (fast correction), small errors produce short micro-pulses (fine adjustment). After each pulse, the system re-evaluates the centroid before issuing the next command — creating a discrete, sample-and-actuate loop.

Typical Tuning Parameters

Dead zone radius	40 pixels
Proportional gain k_p	≈ 0.003 ms/px
Confirmation window	3 consecutive frames
Pulse pause between iterations	80 ms (camera settling time)

Distance Estimation and Approach Phase

§4.1 Monocular Depth via Pixel Area

Once aligned, the platform drives forward toward the object. Without depth sensors, the system exploits a fundamental property of perspective projection: as the robot closes the distance d to the object, its apparent image area A grows inversely with the square of the distance:

$$A \propto \frac{1}{d^2}$$

An empirical threshold $A_{\text{target}} \geq 15,000 \text{ px}^2$ was determined through calibration runs: at this apparent area, the object’s physical position corresponds to the arm’s pre-calibrated optimal grasp point. The robot drives forward in short bursts, re-evaluating the area after each burst.

§4.2 ZUPT-Inspired Stall Detection and Recovery

4.2.1 The Stall Problem

Mobile platforms frequently stall on high-friction surfaces (carpet, rubberised mats) at the lower PWM values used during the precise approach phase. A stall causes the pipeline to wait indefinitely for A to reach A_{target} — a deadlock.

4.2.2 Rolling-Window Detection

Inspired by **Zero Velocity Update (ZUPT)** algorithms from inertial navigation systems, the pipeline maintains a rolling window of the last N area measurements $\{A_{t-N}, \dots, A_t\}$. If the motors are active but:

$$\Delta A = A_t - A_{t-N} < \delta_{\min}$$

...the system infers that no forward movement has occurred: a **physical stall is flagged**.

4.2.3 Escalating Recovery Protocol

Rather than a single recovery attempt, the pipeline uses a two-tier escalating protocol:

1. **Soft Boost:** Motor PWM is briefly spiked to 60% for 300 ms. This overcomes mild static friction without risk of overshooting the target.
2. **Hard Unstick:** If the stall persists after the soft boost, the robot executes a 200 ms reverse jog followed by a high-torque forward lunge at 80% PWM. This breaks deep static friction by first un-loading the contact patch before re-engaging.

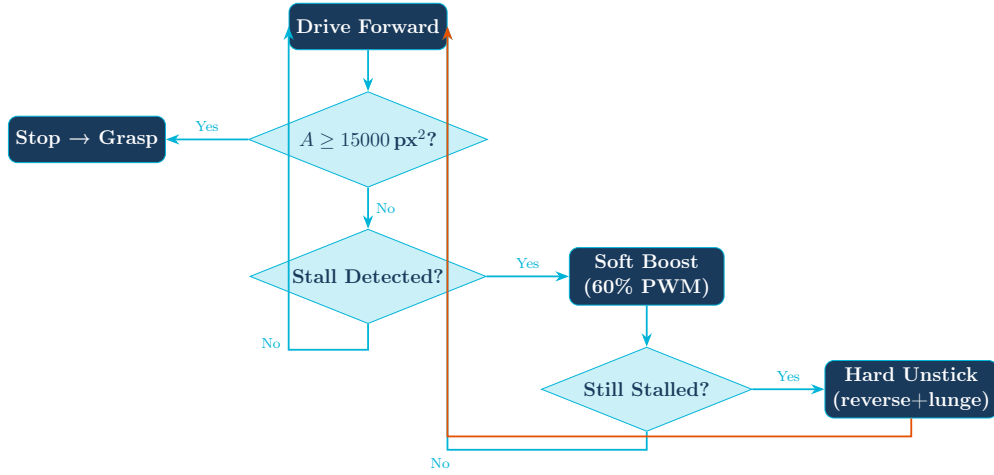


Figure 4.1: Approach phase state machine with ZUPT-inspired stall recovery.

Robotic Arm Kinematics and Manipulation

§5.1 Architecture: Pre-Calibrated Poses vs. Dynamic IK

The 4-DOF robotic arm could theoretically use dynamic Inverse Kinematics (IK) to compute joint angles for arbitrary target positions. However, this approach has a critical weakness on the hardware used: **budget servo motors without absolute encoders**. Without knowing the true current joint angle (only the last commanded angle), IK error compounds across joints, causing the end-effector to miss the target by several centimetres.

The solution is a **hardened state-machine** approach: every critical manipulation point is pre-calibrated and stored as an explicit set of four joint angles (q_{base} , q_{shoulder} , q_{elbow} , q_{gripper}). The IK problem is solved *once* during system calibration and “baked” into the sequence — zero runtime error.

§5.2 Joint Convention

The arm has four degrees of freedom:

Joint	Servo	Function
Base	SG90	Horizontal pan (yaw). In this implementation, the mobile base handles yaw; the base servo is reserved for fine lateral correction.
Shoulder	MG995	Vertical elevation of the upper arm.
Elbow	MG995	Extension/retraction of the forearm.
Gripper	SG90	Open (0°) / Close (70°).

Table 5.1: Arm joint definitions and servo assignments.

§5.3 The Manipulation Sequence

The arm traverses five named states, each with a specific mechanical rationale:

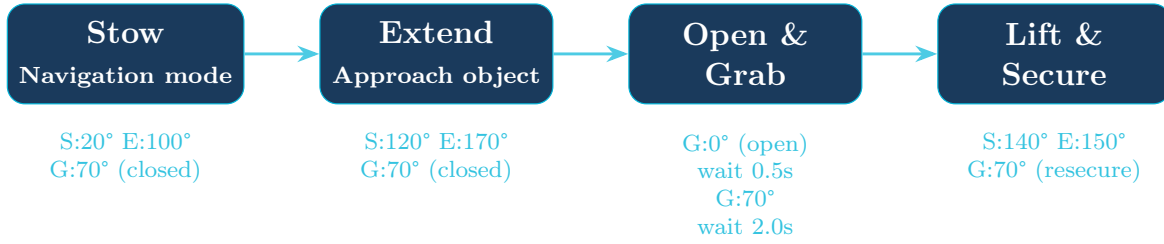


Figure 5.1: Four-state manipulation sequence with calibrated joint angles.

5.3.1 State 1 — Stow (Navigation Mode)

During the entire navigation phase, the arm is tucked close to the chassis: Shoulder at 20°, Elbow at 100°. This serves two critical purposes:

- The arm does not obstruct the camera’s field of view (no self-occlusion).
- The arm’s low centre of mass keeps the platform stable during turning manoeuvres.

5.3.2 State 2 — Extend (Elevated Approach)

The shoulder elevates to 120° *before* the elbow extends to 170°. This **elevated approach trajectory** is essential: if the elbow were to extend horizontally at a low shoulder angle, the arm would sweep across the floor and physically knock the target object away from its calibrated position. The elevated approach ensures the arm descends onto the object vertically, not horizontally.

5.3.3 State 3 — Open and Grab

The gripper opens to 0°, then immediately commands a close to 70°. A deliberate 2.0-second delay is inserted after the close command. This accounts for the mechanical response time of the servo under load: a servo commanded to close against an object draws significantly more current and takes longer to reach full torque than when closing freely. The delay guarantees the gripper has achieved maximum clamping force before the lift begins.

Removing or reducing the 2.0s delay risks attempting the lift before the gripper has fully engaged. At minimum torque, a 100 g object will slip free during the shoulder elevation — losing the object after a successful approach.

5.3.4 State 4 — Lift and Secure

Shoulder elevates to 140° to raise the payload. Simultaneously, the elbow retracts slightly to 150°, shifting the payload’s projected centre of mass *toward* the robot’s

wheelbase centre. This prevents tip-forward instability. A final gripper close command (70°) is re-issued to counteract any slip that may have occurred during the lift acceleration.

Hardware Abstraction Layer

§6.1 The Smooth Motion Problem

Sending a servo from 20° to 140° with a single PWM command causes a violent, instantaneous mechanical snap. On the lightweight arm chassis used, this induces:

- Chassis rocking that can dislodge the grasped object.
- Servo gear damage from peak torque spikes.
- Potential payload drop from inertial forces exceeding grip friction.

§6.2 `arm_servo_listener.py`: The `pigpio` HAL

A dedicated listener process runs on the Raspberry Pi, managing all servo hardware through the `pigpio` daemon. `pigpio` uses hardware DMA (Direct Memory Access) to generate servo PWM signals with microsecond accuracy — far more precise than software PWM approaches.

The listener subscribes to the ZMQ arm-command socket and, upon receiving a target pose, executes the `smooth_move()` algorithm.

6.2.1 `smooth_move()` — Linear Interpolation Profile

The algorithm computes a linearly interpolated trajectory between the current hardware state and the target state:

$$q_i(t) = q_i^{\text{current}} + \frac{t}{T_{\text{total}}} \cdot (q_i^{\text{target}} - q_i^{\text{current}})$$

At each step t , all four joints are updated simultaneously, stepping a maximum of 2° per joint every 20 ms. This gives a maximum angular speed of 100°/s per joint — fast enough for practical use, slow enough to prevent mechanical shock.

The total travel time T_{total} (in ms) for a given pose transition is:

$$T_{\text{total}} = \frac{\max_i |q_i^{\text{target}} - q_i^{\text{current}}|}{2} \times 20 \text{ ms}$$

```

1 def smooth_move(target_angles, current_angles, pi_gpio):
2     """
3     Linearly interpolate all joints to target_angles.
4     Max step: 2 deg/joint per 20 ms iteration.
5     """
6     delta = {j: target_angles[j] - current_angles[j] for j in JOINTS}
7     max_steps = max(abs(delta[j]) for j in JOINTS) // 2 + 1
8
9     for step in range(1, int(max_steps) + 1):
10        for joint in JOINTS:
11            angle = current_angles[joint] + (delta[joint] *
12                step / max_steps)
13            pulse = angle_to_pulse(angle)          # deg ->
14                microseconds
15            pi_gpio.set_servo_pulsewidth(PINS[joint], pulse)
16            time.sleep(0.020)    # 20 ms tick
17
18        # Confirm final position
19        for joint in JOINTS:
20            pulse = angle_to_pulse(target_angles[joint])
21            pi_gpio.set_servo_pulsewidth(PINS[joint], pulse)

```

Listing 6.1: smooth_move() core loop (simplified).

§6.3 PWM Signal Specification

Standard RC servo signals use a 50 Hz PWM carrier with pulse widths mapping angle to position:

Angle (°)	Pulse Width (μ s)	Servo Position
0°	500	Full CW
90°	1500	Centre
180°	2500	Full CCW

Table 6.1: Standard servo PWM mapping.

The conversion used in `angle_to_pulse()` is:

$$\text{pulse} = 500 + \frac{\text{angle}}{180} \times 2000 \quad [\mu\text{s}]$$

Full System Integration and State Machine

§7.1 Master State Machine

All subsystems are orchestrated by a master finite state machine (FSM) running on the PC. The FSM enforces strict phase ordering and handles all error/timeout transitions:

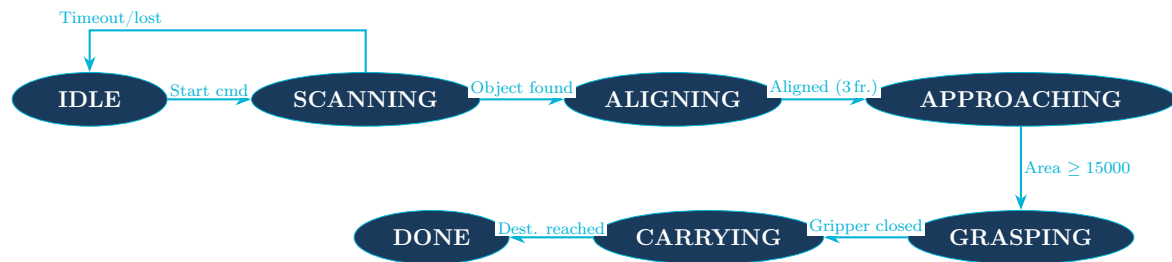


Figure 7.1: Master finite state machine of the retrieval pipeline.

§7.2 Key Design Decisions Summary

Decision	Engineering Rationale
HSV over RGB detection	Illumination-robust colour separation.
Area-based depth	No additional sensors; works within monocular camera budget.
Pre-calibrated poses	Eliminates accumulated IK error from encoder-less servos.
ZUPT stall detection	Detects zero-progress without wheel odometry or IMU.
Proportional pulse control	Simple, effective P-controller; no integral windup risk.
pigpio DMA PWM	Microsecond-accurate servo signals; no jitter from OS scheduling.
Linear interpolation profile	Eliminates mechanical shock; protects servos and payload.
3-frame confirmation	Filters transient centroid jitter; prevents premature approach.
ZMQ CONFLATE socket	Ensures vision pipeline operates on current frames only.
HTTP for motor control	Stateless; missed command isolated; built-in failure recovery.

Table 7.1: Summary of key engineering design decisions.

Hardware Specifications

Component	Model/Spec	Role in System
Edge Processor	Raspberry Pi 3B	All hardware interfacing, motor + servo control, camera capture.
Camera	IMX219 (Pi Camera V2)	8 MP, 1080p30. Primary perception sensor.
Motor Drivers	Dual L298N H-Bridge	4-wheel differential drive; PWM speed + direction control.
Drive Motors	DC Gear Motors $\times 4$	6V, 200 RPM; 4WD chassis.
Shoulder / Elbow	MG995 Servo $\times 2$	High-torque (9.4 kg $\cdot$ cm) arm joints.
Base / Gripper	SG90 Micro Servo $\times 2$	Lightweight positioning joints.
Central Processor	PC (any modern laptop)	OpenCV vision, state machine, Flask HTTP client, ZMQ broker.
Network	Wi-Fi 802.11n	Pi hotspot mode; PC connects directly.
Power (Pi)	5V / 3A USB-C	Dedicated regulator; isolated from motor supply.
Power (Motors)	7.4V LiPo 2S	Provides clean motor voltage separate from logic rail.

Table 8.1: Full hardware specification of the Smart Home & Assistive Robot platform.

Conclusion and Perspectives

§9.1 Summary

This document presents the complete technical design of the **Autonomous Object Retrieval Pipeline**, a subsystem of the Smart Home & Assistive Robot project. The pipeline demonstrates that a fully autonomous pick-and-place cycle — detection, alignment, approach, and grasping — is achievable on severely resource-constrained edge hardware through careful engineering at every stage:

- Classical HSV computer vision for illumination-robust, sub-5 ms object detection.
- Proportional closed-loop alignment with dead-zone anti-hunting and frame-count confirmation.
- Monocular pixel-area distance estimation eliminating the need for depth sensors.
- ZUPT-inspired rolling-window stall detection with an escalating two-tier recovery protocol.
- Pre-calibrated pose sequences eliminating accumulated IK error from encoder-less servos.
- A linear interpolation motion profile preventing mechanical shock during arm transitions.

§9.2 Perspectives and Future Work

1. **Multi-Object Detection:** Replacing single-colour HSV with lightweight feature descriptors (ORB + homography) to detect arbitrary, pre-registered objects.
2. **Dynamic IK with Encoder Feedback:** Adding magnetic angle encoders (AS5600) on MG995 joints to enable real-time IK with closed-loop joint control.
3. **SLAM Integration:** Coupling this pipeline with the ORB-SLAM3 visual odometry layer (already developed on the platform) to enable autonomous long-range navigation to the target before the retrieval pipeline is invoked.
4. **ROS 2 Migration:** Migrating from the custom ZMQ + Flask stack to ROS 2 nodes for standardised inter-process communication, actionlib task management,

and wider community tool support.

5. **Obstacle Avoidance:** Adding an ultrasonic or ToF sensor tier to the approach phase to allow the robot to navigate around obstacles en route to the target zone.

Smart Home & Assistive Robot System

ESI-SBA — Spécialité ISI — 2^{ème} CS

Encadrant : **Pr. Rahmoune**

Rabahi Mohamed Fouad